Tipos de Datos Fundamentales y Avanzados en PostgreSQL

Guía Técnica: Numéricos, Caracteres, Fechas, JSONB, Arrays y Direcciones de Red

Jaqueline Diaz Velasco
Documentación Técnica de Bases de Datos

october 2026

Resumen

PostgreSQL destaca por poseer uno de los sistemas de tipos de datos más ricos y flexibles entre los sistemas gestores de bases de datos relacionales (RDBMS), además de soportar los tipos ANSI SQL estándar como numéricos, caracteres y fechas, incluye tipos avanzados como JSON binario (`JSONB`), arreglos nativos (`ARRAY`) y tipos de datos de red (`INET`/`CIDR`). Este documento detalla la sintaxis, rangos, casos de uso y ejemplos en SQL para cada categoría.

1 Tipos de Datos Numéricos

PostgreSQL ofrece tipos numéricos de tamaño fijo, precisión exacta y precisión flotante.

Tipo	Tamaño	Rango / Descripción	Uso Típico
`SMALLINT`	2 bytes	$-32,768$ a $+32,767$	Códigos pequeños, estados
`INTEGER`	4 bytes	$-2,147,483,648$ a $+2,147,483,647$	IDs estándar, cantidades
`BIGINT`	8 bytes	Generadores de IDs masivos	Contadores de alto tráfico
`NUMERIC(p,s)`	Variable	Exacto (hasta 1000 dígitos)	Moneda, cálculos financieros
`REAL`	4 bytes	Flotante de 6 dígitos de precisión	Mediciones científicas
`DOUBLE PRECISION`	8 bytes	Flotante de 15 dígitos de precisión	Cálculo numérico continuo
`SERIAL` / `BIGSERIAL`	4 / 8 bytes	Autoincremental implícito	Claves primarias

Cuadro 1: Resumen de tipos de datos numéricos.

Nota sobre `NUMERIC(p, s)`: p (precisión) es el número total de dígitos y s (escala) es el número de dígitos a la derecha del punto decimal. Ejemplo: `NUMERIC(10, 2)` permite hasta 8 dígitos enteros y 2 decimales.

2 Tipos de Datos de Carácter

PostgreSQL dispone de tres tipos principales para cadenas de texto:

- `VARCHAR(n)` **(Character Varying):** Cadena de longitud variable con límite de n caracteres. Si se sobrepasa n, PostgreSQL arroja un error.
- `CHAR(n)` **(Character):** Cadena de longitud fija. Si el texto es menor que n, se rellenan espacios en blanco a la derecha. Generalmente se desaconseja salvo para códigos de longitud constante (ej. códigos ISO de país).
- `TEXT`**:** Cadena de texto de longitud ilimitada (hasta 1 GB por campo). En PostgreSQL no existe penalización de rendimiento al usar `TEXT` en comparación con `VARCHAR`.

3 Tipos de Datos de Fecha y Hora

Permiten almacenar instantes temporales, fechas puras o intervalos de tiempo con soporte completo para zonas horarias.

- `DATE`**:** Almacena solo la fecha (año, mes, día). Rango: 4713 a.C. a 5874897 d.C.
- `TIME [WITHOUT | WITH] TIME ZONE`**:** Almacena la hora del día (hora, minuto, segundo, microsegundos).
- `TIMESTAMP [WITHOUT | WITH] TIME ZONE`**:** Almacena fecha y hora completas. `TIMESTAMPTZ` (*with time zone*) convierte automáticamente la fecha ingresada a UTC para almacenamiento y la convierte a la zona horaria de la sesión al consultar.
- `INTERVAL`**:** Representa una duración de tiempo (ej. `'3 days 4 hours'`).

4 Otros 3 Tipos de Datos Fundamentales en PostgreSQL

4.1 1. Documentos Semiestructurados: `JSON` y `JSONB`

PostgreSQL permite trabajar con estructuras JSON nativas.

- `JSONB` **(Binary JSON):** Almacena los datos en formato binario descompuesto. Es ligeramente más lento al insertar pero significativamente más rápido al procesar y consultar, además de admitir la creación de índices GIN.

4.2 2. Colecciones Multi-valor: `ARRAY`

PostgreSQL permite definir columnas como arreglos multidimensionales de cualquier tipo escalar básico (ej. `TEXT[]`, `INTEGER[]`).

4.3 3. Identificadores Únicos y Red: `UUID` e `INET`

- `UUID`**:** Identificador Único Universal de 128 bits garantizado para entornos distribuidos.
- `INET` **/** `CIDR`**:** Almacena direcciones IPv4 o IPv6 con sus respectivas máscaras de red, validando automáticamente la sintaxis del valor ingresado.

5 Ejemplo Práctico de Definición de Tabla en PostgreSQL

El siguiente bloque de código demuestra la integración de los diversos tipos de datos en una sola estructura relacional:

```
CREATE TABLE clientes_avanzado (
    -- Tipos Numericos y Clave Primaria Autoincremental
    id BIGSERIAL PRIMARY KEY,
    saldo_cuenta NUMERIC(12, 2) DEFAULT 0.00,
    puntuacion_credito INTEGER CHECK (puntuacion_credito >= 0),

    -- Tipos de Caracter
    codigo_pais CHAR(3) NOT NULL,
    nombre_completo VARCHAR(150) NOT NULL,
    biografia TEXT,
```

```
    -- Tipos de Fecha y Hora
    fecha_nacimiento DATE NOT NULL,
    creado_en TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    tiempo_sesion INTERVAL,

    -- Tipos Especiales (UUID, JSONB, ARRAY, INET)
    uuid_cliente UUID DEFAULT gen_random_uuid(),
    preferencias JSONB DEFAULT '{"tema": "oscuro", "notificaciones": true}',
    etiquetas TEXT[] DEFAULT ARRAY['nuevo', 'verificado'],
    ultima_ip INET
);
```

6 Conclusión

La diversidad de tipos de datos en PostgreSQL reduce significativamente la necesidad de librerías externas o capas complejas en el backend. La capacidad de mezclar tablas relacionales tradicionales con atributos analíticos (`NUMERIC`), documentos estructurados (`JSONB`), arreglos (`ARRAY`) e identificadores distribuidos (`UUID`) posiciona a PostgreSQL como un sistema gestor de base de datos objeto-relacional versátil y robusto.